

SISTEMAS OPERATIVOS

Conferencia 4 · Tema II: Comunicación entre Procesos y Planificación

Carrera: Ingeniería Informática	Profesora: Ing. Claudia Vecino Fernández
Año: 3ro Período: 2do Tipo: CRD	Correo: claudiavf@uho.edu.cu
Disciplina: Infraestructura de Sistemas Informáticos	Teléfono: 54196249

Objetivo: Comprender los mecanismos de comunicación entre procesos (IPC) — paso de mensajes, tuberías, memoria compartida — y dominar los algoritmos de planificación del procesador: FCFS, SJF, prioridades, Round Robin, colas multinivel y retroalimentación. Analizar la planificación en sistemas operativos actuales.

Sumario

- El problema de los filósofos comensales.
- Comunicación entre procesos (IPC): modelos y mecanismos.
- Paso de mensajes: comunicación directa e indirecta (buzones).
- Capacidad del enlace: cero, limitada, ilimitada.
- IPC moderno: pipes, sockets, memoria compartida, señales.
- Planificación de procesos: conceptos y criterios de evaluación.
- Tipos de planificadores: largo plazo, corto plazo, mediano plazo.
- Algoritmos: FCFS, SJF, Prioridades, Round Robin, Colas Multinivel.
- Planificación en SO actuales: Linux, Windows, macOS.

Bibliografía

- Tanenbaum, A. S. Sistemas Operativos Modernos. Págs. 47–80.
- Tanenbaum, A. S. Operating System: Design and Implementation. Págs. 51–79.
- Silberschatz, Galvin, Gagne. Operating System Concepts, 10ª ed. Caps. 3–5.
- Love, R. Linux Kernel Development, 3ª ed. Cap. 4 (Process Scheduling).

1. El Problema de los Filósofos Comensales

Este problema clásico, propuesto por Dijkstra, modela situaciones donde múltiples procesos compiten por un conjunto limitado de recursos compartidos. Cinco filósofos se sientan alrededor de una mesa con un tenedor entre cada par. Cada filósofo alterna entre pensar y comer; para comer necesita los dos tenedores adyacentes.

El problema de los filósofos comensales es un caso de estudio fundamental para analizar el deadlock (abrazo fatal) y la inanición (starvation) en sistemas concurrentes, ya que una solución ingenua donde cada filósofo toma primero el tenedor de su izquierda provoca deadlock.

La solución con semáforos utiliza dos estructuras: el arreglo `state[]` que refleja el estado de cada filósofo (THINKING, HUNGRY, EATING) y el arreglo de semáforos `fil[]` para el autobloqueo. Un semáforo mutex protege el acceso a estos arreglos.

```
#define N      5          /* número de filósofos */
#define THINKING 0
#define HUNGRY  1
#define EATING  2
#define LEFT    ((i + N - 1) % N) /* vecino izquierdo */
#define RIGHT   ((i + 1) % N)    /* vecino derecho */

int state[N];          /* estado de cada filósofo */
semaphore mutex = {1}; /* exclusión mutua en state[] */
semaphore fil[N];      /* autobloqueo: value=0 inicialmente */

void filosofo(int i) {
    while(1) {
        pensar();
        tomar_tenedores(i); /* puede bloquearse */
        comer();
        soltar_tenedores(i);
    }
}

void tomar_tenedores(int i) {
    P(&mutex);
    state[i] = HUNGRY;
    test(i);          /* intentar comer */
    V(&mutex);
    P(&fil[i]);        /* bloquearse si no pudo comer */
}

void soltar_tenedores(int i) {
    P(&mutex);
    state[i] = THINKING;
    test(LEFT);        /* despertar vecino izquierdo si puede comer */
    test(RIGHT);       /* despertar vecino derecho si puede comer */
    V(&mutex);
}

void test(int i) {
    if (state[i] == HUNGRY &&
        state[LEFT] != EATING &&
        state[RIGHT] != EATING) {
        state[i] = EATING;
        V(&fil[i]);    /* despertar al filósofo i */
    }
}
```

 **Nota actualizada:** Esta solución garantiza ausencia de deadlock y permite el máximo paralelismo (filósofos no adyacentes pueden comer simultáneamente). Sin embargo, puede producir inanición si los vecinos se coordinan para comer alternadamente. Para evitarla se puede añadir un semáforo de turno o usar monitores. En Linux moderno, este patrón se implementa con `pthread_mutex` y `pthread_cond`.

2. Comunicación entre Procesos (IPC)

Los procesos que se ejecutan concurrentemente necesitan con frecuencia comunicarse para mantener una cooperación adecuada. La sincronización entre procesos no es más que un caso particular de esta problemática más general.

Modelo	Descripción
Memoria compartida	Los procesos leen/escriben en una región de memoria común. Requiere sincronización explícita (semáforos, mutexes). Alta velocidad; limitado a un mismo equipo.
Paso de mensajes	Los procesos se comunican enviando y recibiendo mensajes a través del SO. No requiere memoria compartida; funciona en sistemas distribuidos.

Mientras que el uso de memoria compartida queda a la acción de los programadores, los sistemas de intercambio de mensajes son responsabilidad del sistema operativo.

2.1 Paso de Mensajes: Operaciones Básicas

Un sistema de mensajes proporciona dos operaciones fundamentales implementadas como llamadas al sistema:

Primitiva	Semántica
send(dest, msg)	Envía el mensaje msg al destinatario dest. El emisor puede continuar (no bloqueante) o esperar confirmación (bloqueante).
receive(src, &msg)	Recibe un mensaje de src. Si no hay mensaje disponible, el proceso se bloquea hasta que llegue uno.

2.2 Capacidad del Enlace

Un aspecto clave en el diseño de un sistema de mensajes es la capacidad del enlace: la cantidad de mensajes que pueden residir temporalmente en él (implementada como una cola de mensajes):

Capacidad	Comportamiento del emisor	Casos de uso
Cero (Rendezvous)	El emisor se bloquea hasta que el receptor esté listo para recibir. Sincronización garantizada implícitamente.	Protocolos síncronos; Ada rendezvous; llamadas a procedimiento remoto (RPC).
Limitada (cola N)	El emisor continúa si hay espacio; se bloquea si la cola está llena.	Productor/consumidor con buffer acotado; pipes UNIX; MPI.
Ilimitada	El emisor nunca se bloquea; si el receptor no consume, los mensajes se acumulan.	Logs asíncronos; sistemas de eventos; colas de trabajo (message queues).

2.3 Comunicación Directa

Cada proceso que desea enviar o recibir un mensaje debe indicar explícitamente el nombre del otro proceso. Las primitivas adoptan la forma:

```
send(P, mensaje);          /* enviar a proceso P */
receive(Q, &mensaje);      /* recibir del proceso Q */

/* Variante asimétrica: solo el emisor nombra al receptor */
send(P, mensaje);          /* enviar a P */
receive(&id, &mensaje);     /* recibir de cualquiera; id ← PID del emisor */
```

Propiedades del enlace en comunicación directa: se establece automáticamente entre cada par; existe exactamente un enlace por par de procesos; es bidireccional. Desventaja: cambiar el nombre de un proceso obliga a actualizar todas las referencias.

2.4 Comunicación Indirecta: Buzones (Mailboxes)

Los mensajes se envían y reciben de buzones (mailboxes), no directamente entre procesos. Cada buzón tiene un identificador único y puede ser compartido por múltiples procesos.

```
send(buzon_A, mensaje);    /* enviar al buzón A */
receive(buzon_B, &mensaje); /* recibir del buzón B */
```

Propiedad	Comunicación directa vs. indirecta
Establecimiento	Directo: automático entre dos procesos. Indirecto: solo si comparten un buzón.
Número de procesos	Directo: exactamente 2. Indirecto: N procesos pueden compartir un buzón.
Número de enlaces	Directo: uno por par. Indirecto: uno por buzón compartido.
Flexibilidad	Directo: baja (acoplamiento fuerte). Indirecto: alta (desacoplamiento).

Las operaciones que el SO debe proveer sobre buzones son: crear un buzón, enviar/recibir mensajes, y destruir el buzón. Un buzón puede ser propiedad de un proceso (solo él recibe) o del sistema operativo (persistente, compartible).

2.5 IPC Moderno: Mecanismos Actuales

Mecanismo	Descripción	SO / API moderna
Pipes (anónimos)	Canal unidireccional entre proceso padre e hijo. Capacidad limitada (buffer del kernel).	pipe() — POSIX, todos los SO.
Named Pipes (FIFO)	Como pipe pero con nombre en el sistema de ficheros; entre procesos no relacionados.	mkfifo() — POSIX/Linux.
Memoria compartida	Región de memoria mapeada en el espacio de dos o más procesos. Más rápido que mensajes.	shmget()/mmap() — POSIX; CreateFileMapping() — Win32.

Mecanismo	Descripción	SO / API moderna
Sockets	Comunicación bidireccional local o en red. Base de toda comunicación distribuida.	socket()/bind()/connect() — POSIX; BSD Sockets; Winsock.
Señales (Signals)	Notificación asíncrona de eventos. Mecanismo ligero pero limitado en información.	kill()/signal()/sigaction() — POSIX.
Message Queues	Cola de mensajes tipados persistente en el kernel.	mq_open()/mq_send() — POSIX; SysV msgget().
D-Bus / gRPC / ZeroMQ	Frameworks IPC de alto nivel para aplicaciones modernas.	Middleware; independiente del SO.

🔴 **Nota actualizada:** En Linux, el mecanismo de IPC más eficiente para grandes volúmenes de datos es la memoria compartida con mmap() + semáforos POSIX. Para comunicación entre microservicios en contenedores (Docker/Kubernetes) se usan sockets Unix o gRPC. En macOS/iOS, el sistema Mach IPC (ports y mensajes) es la base de toda la comunicación entre procesos.

2.6 Productor/Consumidor con Paso de Mensajes

Utilizando primitivas send/receive se puede resolver el problema del productor/consumidor sin memoria compartida. El consumidor envía N mensajes vacíos al productor al inicio; cada mensaje vacío es un 'permiso' para producir un elemento:

```

/* Productor/Consumidor con paso de mensajes (comunicación directa) */
/* El SO proporciona N buffers de mensajes */

void productor() {
    while(1) {
        producir_elemento(&nextp);
        receive(consumidor, &m);          /* esperar un buffer vacío */
        construir_mensaje(&m, nextp);
        send(consumidor, &m);             /* enviar elemento */
    }
}

void consumidor() {
    for(int i = 0; i < N; i++)             /* inicializar: N buffers vacíos */
        send(productor, &m_vacio);
    while(1) {
        receive(productor, &m);           /* recibir elemento */
        extraer_elemento(&m, &nextc);
        send(productor, &m_vacio);        /* devolver buffer vacío */
        consumir_elemento(nextc);
    }
}

```

💡 **Concepto clave:** La sincronización está garantizada implícitamente: el productor se bloquea en receive si no hay buffers vacíos disponibles; el consumidor se bloquea en receive si no hay elementos producidos. No se necesitan semáforos adicionales.

3. Planificación de Procesos

La planificación del uso de los recursos del sistema constituye una de las funciones fundamentales de los sistemas operativos. El procesador central es el recurso primario de toda computadora y por ello su planificación es esencial.

Definición: El planificador de procesos (Process Scheduler) es el componente del SO que decide qué proceso de la cola de listos recibirá el CPU en cada instante. Los posibles procedimientos que puede utilizar se denominan algoritmos de planificación.

3.1 Tipos de Planificadores

Planificador	También llamado	Función
Largo plazo (Long-term)	Job scheduler	Decide qué trabajos del exterior se admiten en memoria para procesamiento. Controla el grado de multiprogramación. Frecuencia: baja.
Corto plazo (Short-term)	CPU scheduler	Selecciona, de los procesos listos en memoria, al que recibe el CPU. Frecuencia: muy alta (cada interrupción, bloqueo o quantum).
Mediano plazo (Medium-term)	Swapper	Intercambia (swap out/in) procesos entre memoria y disco para mejorar la mezcla de trabajos. Presente en sistemas de memoria virtual.

Otro componente clave es el despachador (Dispatcher): el módulo que realmente entrega el control del CPU al proceso seleccionado por el planificador. Sus funciones incluyen cargar los registros, cambiar a modo usuario y saltar a la dirección de inicio de ejecución. La latencia de despacho (dispatch latency) es el tiempo que toma este cambio de contexto.

3.2 Criterios de Evaluación

Los algoritmos de planificación se evalúan según múltiples criterios que no siempre apuntan en la misma dirección:

Criterio	Descripción	Meta
Eficiencia del CPU	Porcentaje de tiempo en que el CPU ejecuta procesos útiles. Rango real: 40–90%.	Maximizar
Throughput	Número de trabajos completados por unidad de tiempo.	Maximizar
Turnaround	Tiempo total desde que se somete el trabajo hasta que termina (espera en cola + ejecución + E/S).	Minimizar
Tiempo de espera	Tiempo total que el proceso permanece en la cola de listos.	Minimizar

Criterio	Descripción	Meta
Tiempo de respuesta	En sistemas interactivos: tiempo entre la solicitud y el inicio de la respuesta (sin incluir el tiempo de producir la respuesta).	Minimizar
Equidad (Fairness)	Cada proceso recibe una parte razonable del CPU; se evita la inanición.	Garantizar

Las técnicas de planificación se dividen en dos grupos: con derecho de prioridad (preemptive) — el SO puede retirar el CPU a un proceso en ejecución — y sin derecho de prioridad (non-preemptive / cooperativa) — el proceso entrega el CPU voluntariamente al terminar o bloquearse.

4. Algoritmos de Planificación

4.1 FCFS — First Come, First Served

El algoritmo más simple: la atención se brinda según el orden de llegada a la cola de listos. Por naturaleza es sin derecho de prioridad. La cola de listos se opera como una cola FIFO.

Proceso	Ráfaga CPU (ms)	Orden llegada → turnaround
P1	24	Llega primero → turnaround = 24 ms
P2	3	Llega 2do → espera 24 → turnaround = 27 ms
P3	3	Llega 3ro → espera 27 → turnaround = 30 ms
Promedio	—	Turnaround promedio = $(24+27+30)/3 = 27$ ms

⚠ Importante: El efecto convoy: un proceso largo bloquea a todos los que llegan después. Si los procesos llegaran en orden P2→P3→P1, el turnaround promedio sería $(3+6+30)/3 = 13$ ms, mucho mejor. FCFS es sensible al orden de llegada.

4.2 SJF — Shortest Job First

Asigna el CPU al proceso con la menor ráfaga de CPU. Puede ser preemptive (SRTF: Shortest Remaining Time First) o non-preemptive. Es óptimo para minimizar el turnaround y el tiempo de espera promedio cuando todos los trabajos están disponibles simultáneamente.

Variante	Preemptive	Problema principal
SJF non-preemptive	No — el proceso en ejecución no se interrumpe.	Difícil conocer de antemano la duración.
SRTF (preemptive)	Sí — llega un proceso más corto → preempt.	Puede causar inanición de procesos largos.

Para aplicar SJF en la planificación a corto plazo se usa predicción exponencial de la próxima ráfaga: $\tau(n+1) = \alpha \cdot t(n) + (1-\alpha) \cdot \tau(n)$, donde $t(n)$ es la duración real de la ráfaga n , $\tau(n)$ es la predicción anterior, y $\alpha \in [0,1]$ controla el peso del historial reciente.

4.3 Planificación por Prioridades

Cada proceso tiene asociada una prioridad; el CPU se asigna al proceso listo de mayor prioridad. Puede ser preemptive o non-preemptive. Las prioridades pueden ser estáticas (externas, definidas por el usuario/administrador) o dinámicas (calculadas por el SO).

Problema	Solución
Inanición (starvation)	Un proceso de alta prioridad que consume mucho CPU puede acaparar el procesador indefinidamente.
Solución: (envejecimiento)	Aging El SO incrementa gradualmente la prioridad de los procesos que llevan mucho tiempo esperando. Garantiza que todos eventualmente sean atendidos.

4.4 Round Robin (RR) — Turno Circular

Diseñado para sistemas de tiempo compartido. La cola de listos se trata como una cola circular. Cada proceso recibe el CPU durante un quantum (time slice) de tiempo; si no termina, se coloca al final de la cola. Siempre es preemptive.

 **Concepto clave:** El quantum es el parámetro crítico del Round Robin: si es muy pequeño, el overhead del cambio de contexto domina y la eficiencia cae; si es muy grande, degenera en FCFS. Valor típico en sistemas actuales: 10–100 ms. Regla práctica: el 80% de las ráfagas deben ser menores que el quantum.

Si hay n procesos en la cola y el quantum es q , cada proceso recibe $1/n$ del tiempo del CPU en bloques de q ms. Ningún proceso espera más de $(n-1) \cdot q$ ms antes de recibir el CPU de nuevo.

4.5 Colas Multinivel (Multilevel Queue)

La cola de listos se divide en múltiples colas, cada una para una clase de procesos. Cada cola tiene su propio algoritmo de planificación y su propia prioridad. Los procesos se asignan permanentemente a una cola según sus características.

Cola	Tipo de proceso	Algoritmo	Prioridad
1	Procesos de sistema (System)	FCFS	Más alta
2	Interactivos (foreground)	Round Robin	Alta
3	Interactivos de edición	Round Robin	Media
4	Procesos por lotes (batch)	SJF/FCFS	Baja
5	Tareas estudiantes / bajo valor	FCFS	Más baja

 **Importante:** Las colas de baja prioridad pueden sufrir inanición si llegan continuamente procesos de alta prioridad. Mitigación: asignar a cada cola un porcentaje del tiempo de CPU (ej: 80% para foreground, 20% para background).

4.6 Colas Multinivel con Retroalimentación (Multilevel Feedback Queue)

Generalización del esquema anterior: los procesos pueden moverse entre colas según su comportamiento de ejecución. Es el algoritmo más general y flexible. Parámetros de diseño: número de colas, algoritmo de cada cola, criterios de ascenso/descenso, y criterio de entrada inicial.

Funcionamiento típico: los procesos nuevos entran en la cola de más alta prioridad (quantum pequeño). Si consumen todo el quantum sin bloquearse, descienden a la cola siguiente (quantum mayor). Los procesos que se bloquean frecuentemente por E/S permanecen en colas altas (son interactivos). Así, los trabajos cortos e interactivos se atienden rápidamente y los largos van bajando de prioridad pero con quantums cada vez mayores.

```
/* Ejemplo: 3 colas con retroalimentación */
Cola Q0: quantum = 8ms    (prioridad más alta, procesos interactivos)
Cola Q1: quantum = 16ms   (prioridad media)
Cola Q2: FCFS              (prioridad más baja, trabajos largos de CPU)
```

Reglas:

- Proceso nuevo → Q0
- Consume todo el quantum en Q0 → baja a Q1
- Consume todo el quantum en Q1 → baja a Q2
- Se bloquea por E/S en cualquier cola → sube a Q0 al desbloquearse
- Proceso en Q2 espera mucho (aging) → sube a Q1

✦ **Nota actualizada:** El algoritmo Multilevel Feedback Queue es la base del planificador de la mayoría de los SO modernos. Linux usa el CFS (Completely Fair Scheduler) desde la versión 2.6.23; Windows utiliza un esquema de 32 niveles de prioridad con feedback dinámico; macOS emplea un planificador por bandas (bands) similar al MFQ.

4.7 Planificación en Multiprocesadores

En sistemas con múltiples CPUs (o núcleos) surgen nuevas consideraciones. Se distinguen dos enfoques principales:

Enfoque	Descripción
Cola única (SMP)	Todos los procesadores comparten una cola de listos global. Requiere exclusión mutua en el acceso a la cola. Buen balance de carga automático.
Cola por procesador	Cada procesador tiene su propia cola. Menor contención. Requiere algoritmo de balance de carga (load balancing) para evitar que un CPU esté ocioso mientras otro está sobrecargado.
Maestro/Esclavo	Un procesador (maestro) planifica para todos los demás (esclavos). Simple pero el maestro es cuello de botella.

✦ **Nota actualizada:** En procesadores modernos multicore con NUMA (Non-Uniform Memory Access), la localidad de caché (cache affinity) es un factor crítico: conviene mantener los procesos en el mismo núcleo que ya tiene sus datos en caché. Linux CFS implementa load balancing con migración de tareas entre dominios de scheduling respetando la topología NUMA.

5. Planificación en Sistemas Operativos Actuales

5.1 Linux — Completely Fair Scheduler (CFS)

El planificador actual de Linux (desde 2.6.23) es el CFS, que reemplazó al planificador O(1) anterior. Su objetivo es asignar a cada proceso una proporción justa del tiempo del CPU, modelando un procesador ideal que ejecuta todos los procesos simultáneamente a velocidad $1/n$.

Característica	Descripción
Estructura de datos	Árbol rojo-negro (red-black tree) ordenado por vruntime (tiempo virtual de CPU consumido).
Selección	Siempre elige el proceso con menor vruntime (el menos favorecido hasta ahora).
Nice values	Prioridades -20 a +19 (nice). Afectan el peso en el cálculo del vruntime.
Clases de scheduling	SCHED_NORMAL (CFS), SCHED_FIFO (RT, time-critical), SCHED_RR (RT round-robin), SCHED_DEADLINE (CBS).
Grupos de control (cgroups)	Permiten aplicar CFS a grupos de procesos (ej: contenedores Docker) garantizando cuotas de CPU.

5.2 Windows — Priority-Based Preemptive Scheduling

Windows utiliza un esquema de planificación por prioridades con 32 niveles (0–31). Los hilos (threads) son la unidad de planificación. Los niveles se dividen en dos categorías: tiempo real (16–31) y dinámicos (1–15).

Característica	Descripción
Quantum variable	Los hilos foreground (con ventana activa) reciben quantum más largos que los background.
Priority boosting	El SO incrementa temporalmente la prioridad de hilos que: (a) salen de espera por E/S, (b) tienen ventana activa, (c) esperan mucho en semáforo.
Unidad de planificación	Thread (hilo), no proceso. Un proceso con múltiples hilos puede tener hilos en distintas colas.
NUMA awareness	El dispatcher de Windows considera la topología NUMA para minimizar accesos remotos a memoria.

5.3 UNIX clásico — Prioridades Dinámicas

El UNIX clásico es un sistema interactivo de tiempo compartido con multiprogramación. Los procesos listos se mantienen en colas multinivel: cada cola corresponde a una prioridad calculada dinámicamente cada segundo, en función del tiempo de CPU consumido.

A medida que un proceso acumula tiempo de CPU va perdiendo prioridad. Los procesos que salen del estado bloqueado (por llamada al sistema, como E/S) reciben una prioridad negativa, lo que los hace más prioritarios — el planificador UNIX beneficia los trabajos interactivos.

SO	Resumen del mecanismo de planificación
Linux (CFS)	Árbol rojo-negro por vruntime. Clases de tiempo real (FIFO/RR) y CFS. cgroups para cuotas.
Windows NT/10/11	32 niveles de prioridad. Priority boosting dinámico. Thread como unidad. NUMA-aware.
macOS (XNU)	4 bandas de prioridad (normal, system, kernel, realtime). Similar a MFQ.
UNIX clásico	Colas multinivel con prioridades dinámicas/segundo. Quantum ~100ms. Favorece interactivos.
MS-DOS (histórico)	Monoprogramado. Sin planificador real: un proceso activo hasta terminar. TSR como excepción.

📌 **Nota actualizada:** En sistemas embebidos y de tiempo real (RTOS como FreeRTOS, VxWorks, Zephyr), el planificador es estrictamente preemptive por prioridades fijas (Rate Monotonic, Earliest Deadline First — EDF). La latencia máxima garantizada es un requisito de diseño fundamental, no opcional.

6. Estudio Independiente

- En el problema de los filósofos comensales, ¿por qué una solución donde cada filósofo toma primero el tenedor de su izquierda provoca deadlock? ¿Cómo lo evita la solución con semáforos presentada?
- Explique las diferencias entre comunicación directa e indirecta (buzones). ¿En qué escenario preferiría usar cada una?
- Compare los tres tipos de capacidad de enlace (cero, limitada, ilimitada) en términos de sincronización y bloqueo. Proporcione un caso de uso real para cada uno.
- Dados los procesos P1(ráfaga=8ms), P2(ráfaga=4ms), P3(ráfaga=1ms), P4(ráfaga=9ms) llegando en ese orden: calcule el turnaround y tiempo de espera promedio con FCFS, SJF non-preemptive y Round Robin (quantum=3ms).
- ¿Por qué el algoritmo SJF puede causar inanición? ¿Cómo el mecanismo de aging (envejecimiento) resuelve este problema?
- Explique el funcionamiento del algoritmo Multilevel Feedback Queue con 3 colas. ¿Qué tipos de procesos permanecen en la cola de mayor prioridad y por qué?
- Investigue el CFS (Completely Fair Scheduler) de Linux: ¿qué es el vruntime? ¿Cómo usa el árbol rojo-negro para seleccionar el siguiente proceso? ¿Qué diferencia hay entre SCHED_NORMAL y SCHED_FIFO?

— Fin de la Conferencia 4 —